

MBAX 6418

LLM Fundamentals: Models, APIs, and Interfaces

David Dobolyi, PhD

Assistant Professor

Stone Family Faculty Scholar

david.dobolyi@colorado.edu



Leeds School of Business
UNIVERSITY OF COLORADO **BOULDER**



Topics

- Large Language Models (LLMs) for Coding
- Vibe Coding Overview
- Vibe Coding Interfaces

LLMs for Coding

Open- vs. Closed-Weight LLMs

- **Open-Weights**

- Free to download
- Run locally or via cloud application programming interfaces (APIs)*
- Reproducible and private (if local)



Hugging Face



OpenRouter



ModelScope



Qwen



deepseek



- **Closed-Weights**

- Higher performing frontier models
- Easier to use/deploy via interfaces and/or APIs
- Variable costs/credit allowances



NVIDIA



MINIMAX



Claude



Gemini

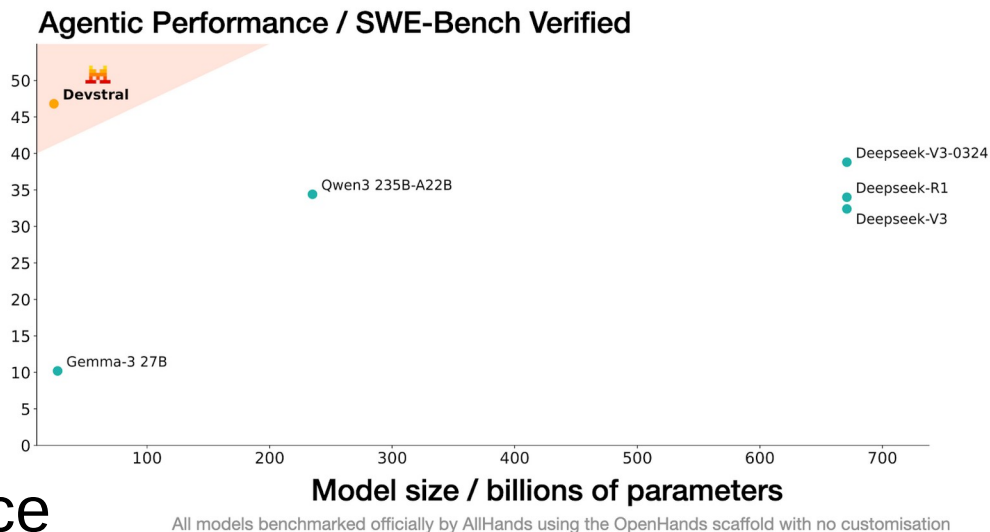
*Open and closed APIs let you send requests to an LLM service and receive model responses programmatically.

Training Data

- Models must be pre-trained on large corpuses of data
- These data may be either open or closed
 - For fully open examples, see [Olmo from Ai2](#) and [NVIDIA Nemotron](#)
- Some model providers have been accused of copyright infringement
 - See [The New York Times Company v. Microsoft Corp., OpenAI, ...](#)
- Models will often include a “knowledge cutoff date” to clarify how up-to-date their training data is (or isn’t)

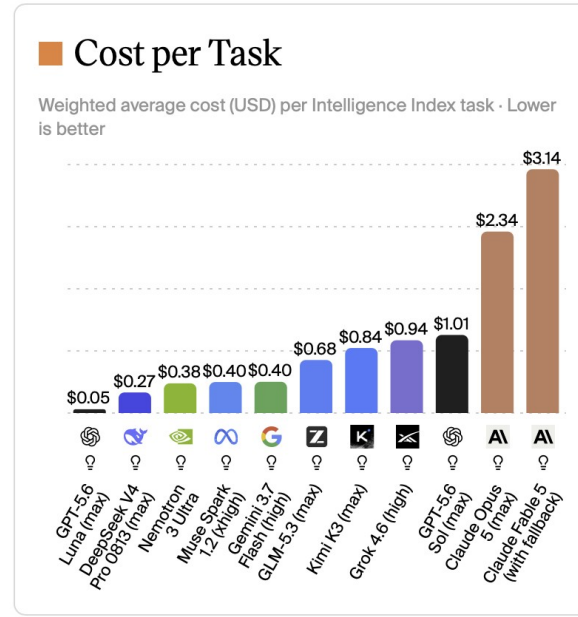
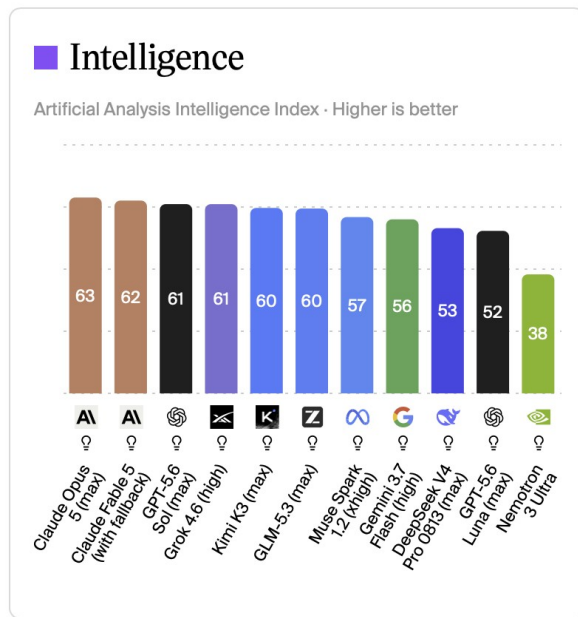
Model Size and “Intelligence”

- Models come in various *parameter sizes*
 - e.g., 30B, 671B
- Larger models tend to perform better, but so do more recent models
- Consider size vs. knowledge vs. performance vs. cost tradeoffs



Benchmark Evaluation

- [SWE-Bench Pro](#)
- [DeepSWE](#)
- [Terminal-Bench](#)
- [Agents' Last Exam](#)
- [AutomationBench](#)
- [Artificial Analysis Intelligence Index](#)



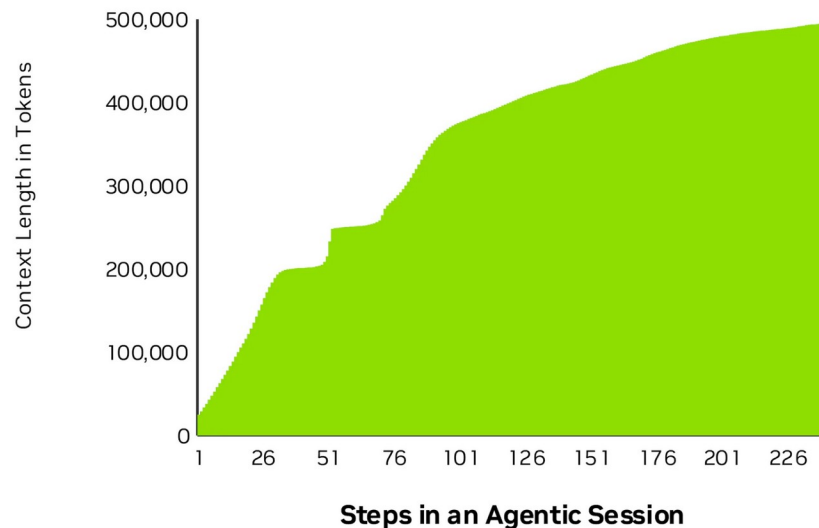
See the AAIL website for an additional chart for “Speed.”

Context, Context, Context...

- Besides “intelligence,” one must consider **context**, or how much text (or other information) a model can access while generating subsequent tokens
- Provides “working memory,” i.e., how much the model can remember and work with before forgetting
- Measured in *tokens* (i.e., text chunks) and referred to as *context length* or *context window*
- Covers both input and output tokens
- **TLDR:** more context = better (especially for coding)

Managing Long Context

- Agentic AI involves long conversations and thus long context
- Context must be compressed, compacted, summarized, and/or pruned over time
- Model accuracy and performance degrades by context length (e.g., see [NVIDIA RULER](#))



Inference Speed

- Model inference is often measured relative to two phases
- The first phase is ***prefill***, or ***prompt processing***, which occurs when your input tokens are processed
 - Typically measured in terms of **time-to-first-token (TTFT)** or prefill **tokens-per-second (tok/s)** at various prompt lengths
- The second phase is ***decode***, or ***token generation***, which involves how quickly output tokens are generated
 - Typically measured in **tokens-per-second (tok/s)**
- These metrics are generally not linear and vary based on factors such as context length

KV Caching

- A single LLM response can involve a sequence of many inference passes (e.g., once per every output token in the response)
- By using a ***key-value (KV) cache***, decoding can move faster by avoiding the need to reprocess prior tokens when generating new ones, reducing TTFT
- The tradeoff of KV cache is increased memory demands

Prompt Caching

- ***Prompt (or prefix) caching*** extends this performance boost by allowing KV cache to be reused across requests
- For example, consider a long agentic session involving a ten messages with 100K tokens in use. The user is about to send their eleventh message of 1K tokens:
 - Without prefix caching, the model must prefill the full 101K tokens before beginning generation
 - With prefix caching, the model can reuse the KV cache for the previous 100K tokens and process only the new 1K tokens, leading to significantly lower TTFT

Speculative Decoding

- ***Speculative decoding*** is a method of “guessing” several upcoming candidate tokens (e.g., 3 tokens ahead) that are verified by the model in a single pass to improve throughput
 - Some methods under this umbrella such as multi-token prediction (MTP) are built into models (e.g., Qwen3.8)
 - Others such as DSpark, DFlash, and DFlash 2 involve a separate *drafter* model that works in conjunction with a larger model
- Benchmarking speculative decoding at a variety of output (i.e., 3 vs. 5 tokens) and context lengths is important, as longer lengths can hinder speedup (also see [Adaptive MTP](#))

Concurrency

- **Concurrency** involves processing multiple prompts/requests simultaneously to increase computational efficiency
 - Can be combined with **continuous batching**, where multiple requests are strategically combined into a single execution step, to further improve performance

Quantization

- ***Quantization*** is a process for scaling and rounding model parameters to make models smaller and potentially faster
- Many models are trained at 16-bit precision (e.g., FP16, BF16), but this can be reduced to 8-bits (e.g., FP8) or 4-bits (e.g., INT4) without significant accuracy degradation
- For example, consider **Qwen3-Coder-30B-A3B-Instruct**
 - [16-bits](#): ~61GB; [8-bits](#): ~31GB; [4-bits](#): ~17GB
- Quantization can also be applied to model context (e.g., [TurboQuant](#)), although this can lead to output quality degradation (particularly below FP8) or a performance hit

Quantization Formats

- Various quantization formats exist to help reduce hardware requirements and improve inference performance
- Use the ones appropriate to your hardware
- Prefer 4-bit (or larger) formats
- Popular options (mid 2026)
 - GPU: [AWQ](#), [GPTQ](#), [NVFP4](#), [FP8](#)
 - Apple M chips: [MLX](#), [oQ+](#)
 - CPU+GPU (includes Apple M chips): [GGUF](#) (e.g., Q4_K_M for 4-bits)

Model Types: Instruct, MoE

- **Instruct (Instruction Tuned)**
 - Tunes a *base* model to take an instruction and provide one response
 - Not necessarily tuned for multi-turn “chat”
- **MoE (Mixture of Experts)**
 - Model consists of several mini-models or “experts”
 - Only some “active” experts will respond to a particular input
 - Performance can be significantly faster (in tokens/sec) than a “dense” model (i.e., non-MoE)

Reading a Model Name

- **Example: Qwen3-Coder-30B-A3B-Instruct-FP8**
 - **Model Family:** Qwen3-Coder-30B-A3B-Instruct-FP8
 - **Model Variant:** Qwen3-Coder-30B-A3B-Instruct-FP8
 - **Model Size/Type:** Qwen3-Coder-30B-A3B-Instruct-FP8
 - ~30B Total Parameters
 - ~3B Active Parameters (i.e., Model is a MoE)
 - **Model Tuning:** Qwen3-Coder-30B-A3B-Instruct-FP8
 - **Quantization:** Qwen3-Coder-30B-A3B-Instruct-FP8
- *Note that these conventions can (and do) vary...*

Other Model Considerations

- **Multimodal**

- Some models work with modalities beyond text, e.g., vision, audio
- These vision-language models (VLMs) may include “VL” or “V” in the model’s name

- **Thinking/Reasoning**

- Some models simulate cognitive processing (e.g., chain-of-thought [CoT]) for internal planning, often allowing for better response quality

- **Tool Calling/Tool Use/Agentic**

- Some models advertise “agentic” abilities, which include setting goals, planning, and acting while using “tools” to complete tasks

Sycophancy

- Be careful with models that are overly “agreeable”
- Models are measured in terms of ***alignment*** to address this concern (e.g., Claude Sonnet 4.5)
- Consider benchmarks such as **SycEval** (Fanous et al. 2025)



Luke Stephens (hakluke) ✓
@hakluke



When asking coding questions



**you're
absolutely
wrong**



ChatGPT

**you're
absolutely
right**

TLDR: How to Choose a Coding Model

- **Context** is often paramount...more context = better
- **Prefer larger models** (and **higher reasoning** settings), but consider tradeoffs...
 - **Pros:** more intelligent; higher output quality
 - **Cons:** slower; require more hardware for local deployment; more expensive in the cloud
- **Additional considerations...**
 - MoE models are often faster than dense models
 - Quantized models offer an efficiency/accuracy tradeoff
 - Not all models accept multimodal inputs, optimize for tool usage, etc.

Popular Open Coding Models (Aug 2026)

Name	Params (B)	Context	Multimodal	Type	Date
GLM-5.3 Flash	320	1M	Yes	MoE	08/26
Qwen3.8-Flash-Next	125*	256K [†]	Yes	MoE	08/26
Qwen3.8-27B	27	256K [†]	Yes	Dense	08/26
Muse Glimmer	30	128K [†]	Yes	Dense	08/26
DeepSeek-V4-Flash-0731	284	1M	No	MoE	07/26
Kimi K3	2800	1M	Yes	MoE	07/26
GLM-5.2	753	1M	No	MoE	06/26
MiniMax M3	428	1M	Yes	MoE	06/26
Gemma 4	Varies	Varies	Yes	Varies	04/26

*Plus 51B n-gram embedding (and 4B MTP)

[†]Can be extended via methods such as YaRN, RoPE

Popular Closed Coding Models (Aug 2026)

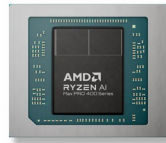
Name	Params (B)	Context	Multimodal	Type	Date
Gemini 3.7 Flash	?	1M	Yes	?	08/26
Claude Opus 5	?	1M	Yes	?	07/26
Grok 4.5	?	500K	Yes	?	07/26
GPT-5.6 Sol/Terra/Luna	?	1.05M*	Yes	?	07/26
SWE-1.7	?	? [†]	No	?	07/26
Claude Sonnet 5	?	1M	Yes	?	06/26
Claude Fable 5	?	1M	Yes	?	06/26
Composer 2.5	?	200K	No	?	05/26

*May be capped in the API at 272K or 872K

[†]Model features self-compaction with unspecified context window

Hardware Requirements for Running Local

- Video memory (e.g., VRAM) and/or (ideally unified) system memory (RAM) dictate max local model size
- You *can* run small models at 16GB, but more = better
- Price-to-performance hardware options (mid 2026)
 - Apple M chips (e.g., MacBook Pro M5 Max 128GB; Mac Studio M5 Ultra 512GB)
 - AMD Ryzen AI Max+ PRO 495 192GB
 - NVIDIA RTX Pro 6000 Blackwell 96GB
 - NVIDIA RTX Spark, DGX Spark 128GB

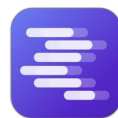


Enterprise Deployment

- For larger scale deployment, consider servers with multiple graphical processing units (GPUs)
- For inference, there are many viable options from AMD, NVIDIA, Intel, etc.
- Moreover, Application-Specific Integrated Circuits (ASICs) are becoming popular for their high performance-per-watt characteristics (e.g., [OpenAI Jalapeño](#) with Broadcom)
- If training or fine-tuning matters, NVIDIA's Compute Unified Device Architecture (CUDA) is currently unparalleled

Running Local Models

- Best choice for efficient inferencing depends on hardware
- Recommendations for most users:
 - Graphical User Interfaces (GUIs): [LM Studio](#), [Ollama](#)*
- Recommendations for technical users:
 - GPU: [vLLM](#), [SGLang](#)
 - Apple M chips: [oMLX](#), [MTPLX](#), [mlx-serve](#)
 - CPU, CPU+GPU†: [llama.cpp](#), [KTransformers](#)



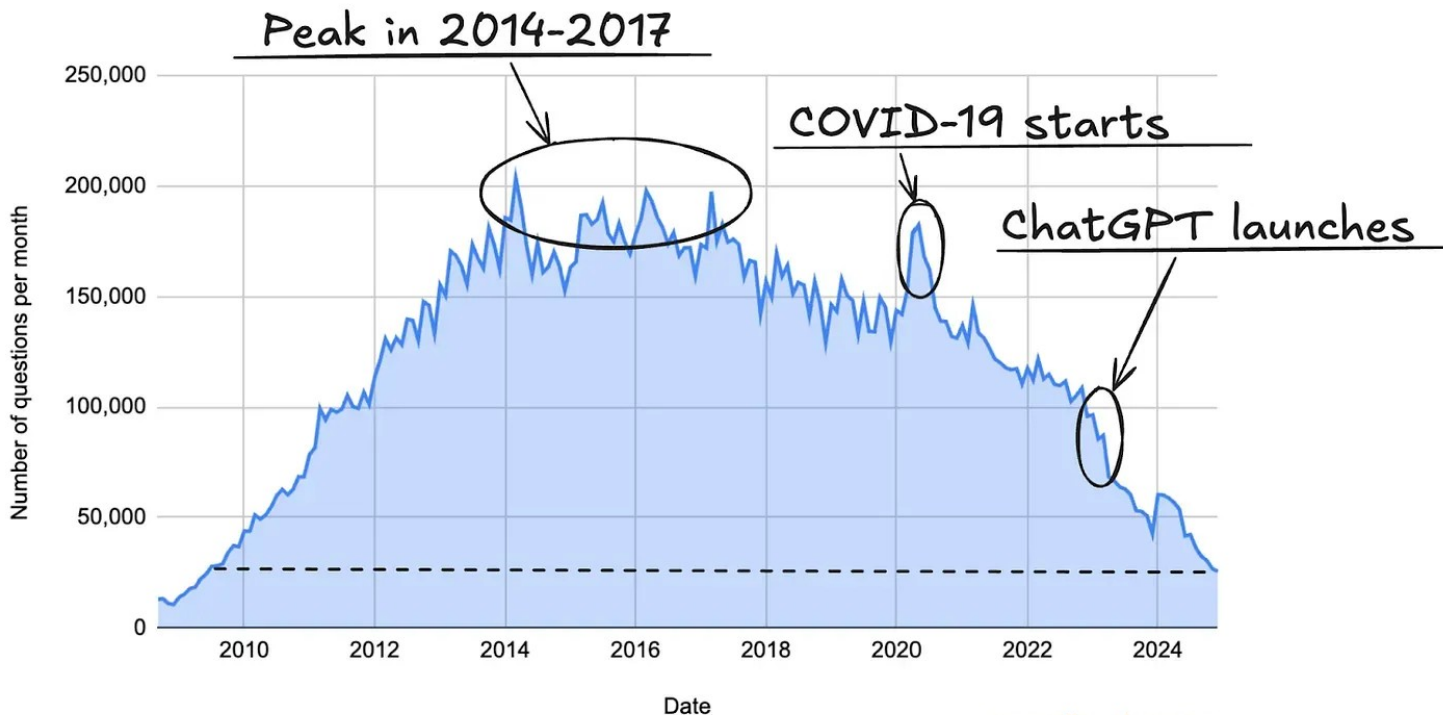
*See slide notes on how to configure default context length limits

†CPU+GPU offloading can increase hardware flexibility, albeit with a performance tradeoff

Vibe Coding Overview

LLMs Changed Things...Forever?

Monthly questions asked on StackOverflow



Vibe Coding



Andrej Karpathy ✓

@karpathy



There's a new kind of coding I call "vibe coding", where you fully give in to the vibes, embrace exponentials, and forget that the code even exists. It's possible because the LLMs (e.g. Cursor Composer w Sonnet) are getting too good. Also I just talk to Composer with SuperWhisper so I barely even touch the keyboard. I ask for the dumbest things like "decrease the padding on the sidebar by half" because I'm too lazy to find it. I "Accept All" always, I don't read the diffs anymore. When I get error messages I just copy paste them in with no comment, usually that fixes it. The code grows beyond my usual comprehension, I'd have to really read through it for a while. Sometimes the LLMs can't fix a bug so I just work around it or ask for random changes until it goes away. It's not too bad for throwaway weekend projects, but still quite amusing. I'm building a project or webapp, but it's not really coding - I just see stuff, say stuff, run stuff, and copy paste stuff, and it mostly works.

6:17 PM · Feb 2, 2025 · 4M Views



1.2K



3.9K



24K

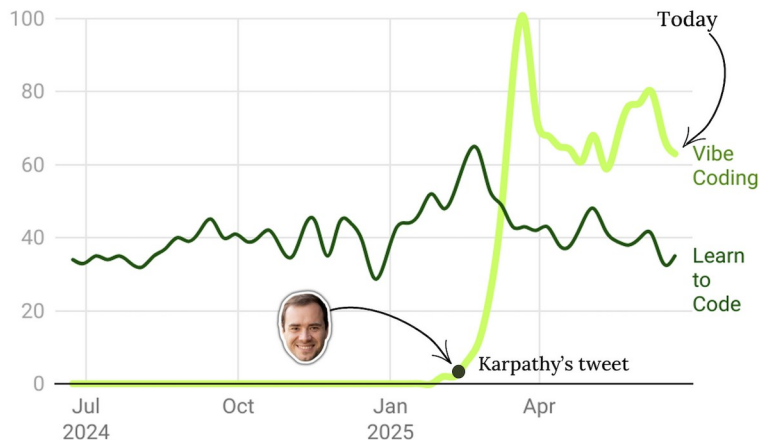


12K



More People Are Searching 'Vibe Coding' Than 'Learn to Code'

Interest Over Time for "Vibe Coding" and "Learn to Code" on Google in the  from July 2024



Source: Google Trends

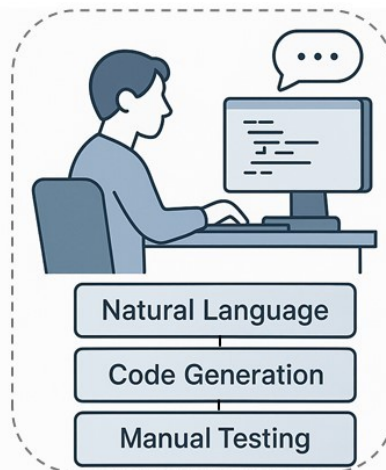
Vibe Coding vs. Agentic Coding

- **Vibe Coding**

- Human-in-the-loop
- Prompt-based
- Conversational
- Designed for intuitive ideation and creative exploration

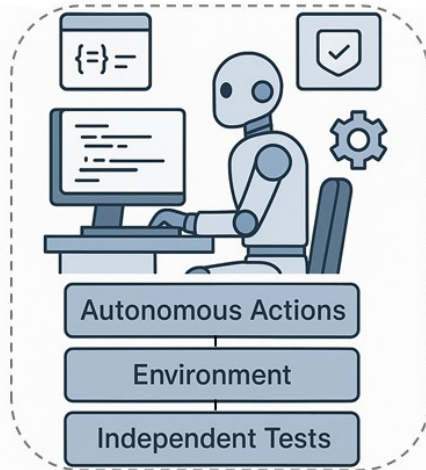
- **Agentic Coding**

- Autonomous
- Involves goal-driven agents that plan, execute, test, and iterate
- Minimal human input



(a)

Vibe Coding

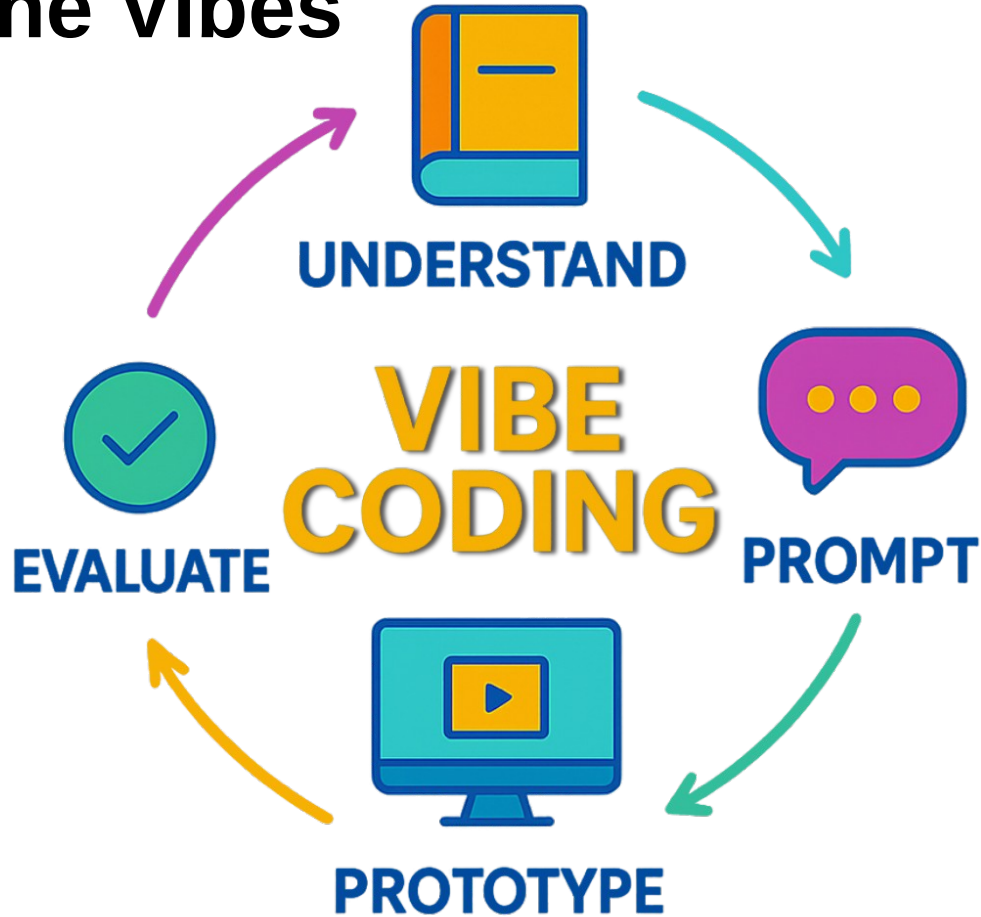


(b)

Agentic Coding

(Sapkota, Roumeliotis, & Karkee 2025)

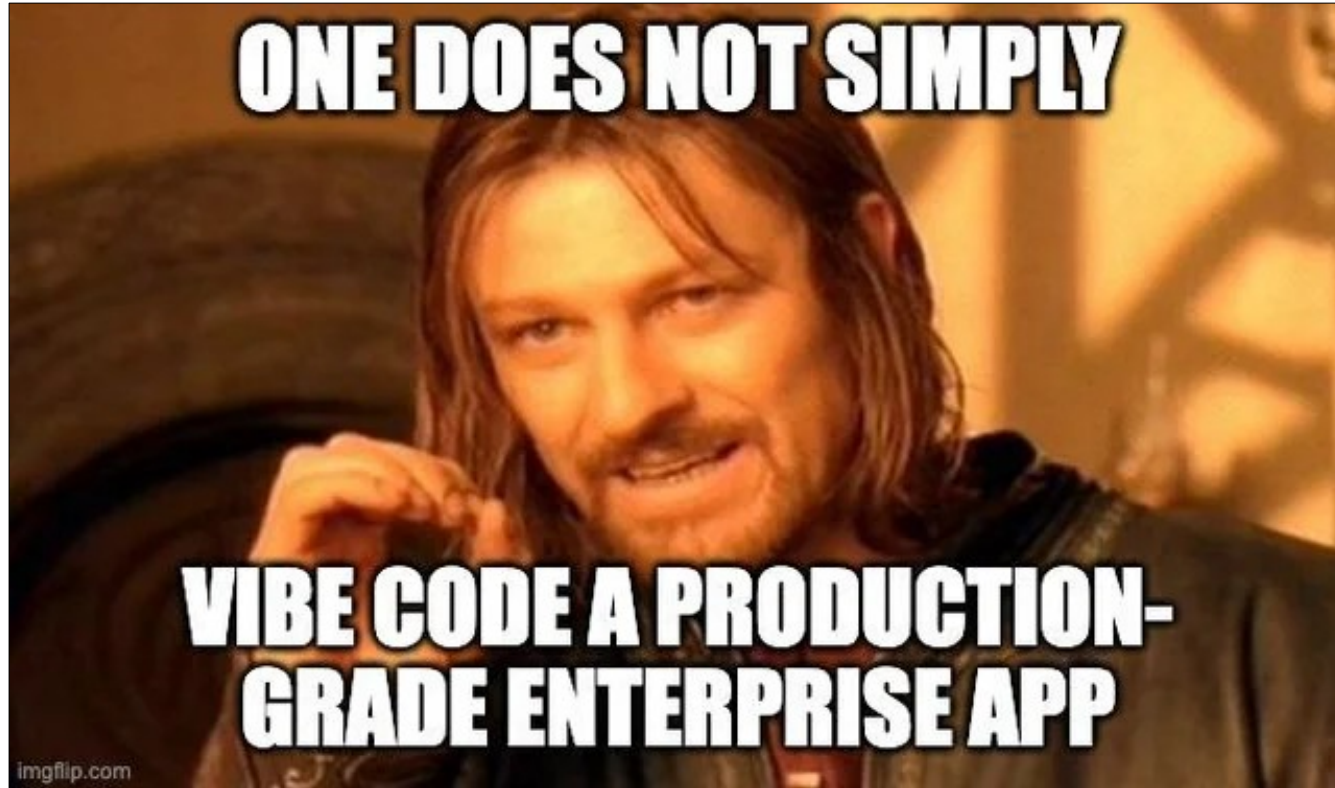
Iterating the Vibes



Recommended Use Cases

- Rapid prototyping
- Designing user interfaces
- Explaining codebases
- Translating code across languages
- Pair programming

Vibe Coding on “Prod”



Security? Never heard of it...

How it started...



leo

@leojr94_

my saas was built with Cursor, zero hand written code

AI is no longer just an assistant, it's also the builder

Now, you can continue to whine about it or start building.

P.S. Yes, people pay for it

4:34 am · 15 Mar 2025 · **52.2K** Views

How it's going...



leo

@leojr94_

guys, i'm under attack

ever since I started to share how I built my SaaS using Cursor

random thing are happening, maxed out usage on api keys, people bypassing the subscription, creating random shit on db

as you know, I'm not technical so this is taking me longer that usual to figure out

for now, I will stop sharing what I do publicly on X

there are just some weird ppl out there

9:04 am · 17 Mar 2025 · **53.6K** Views

Vibe Coding Interfaces

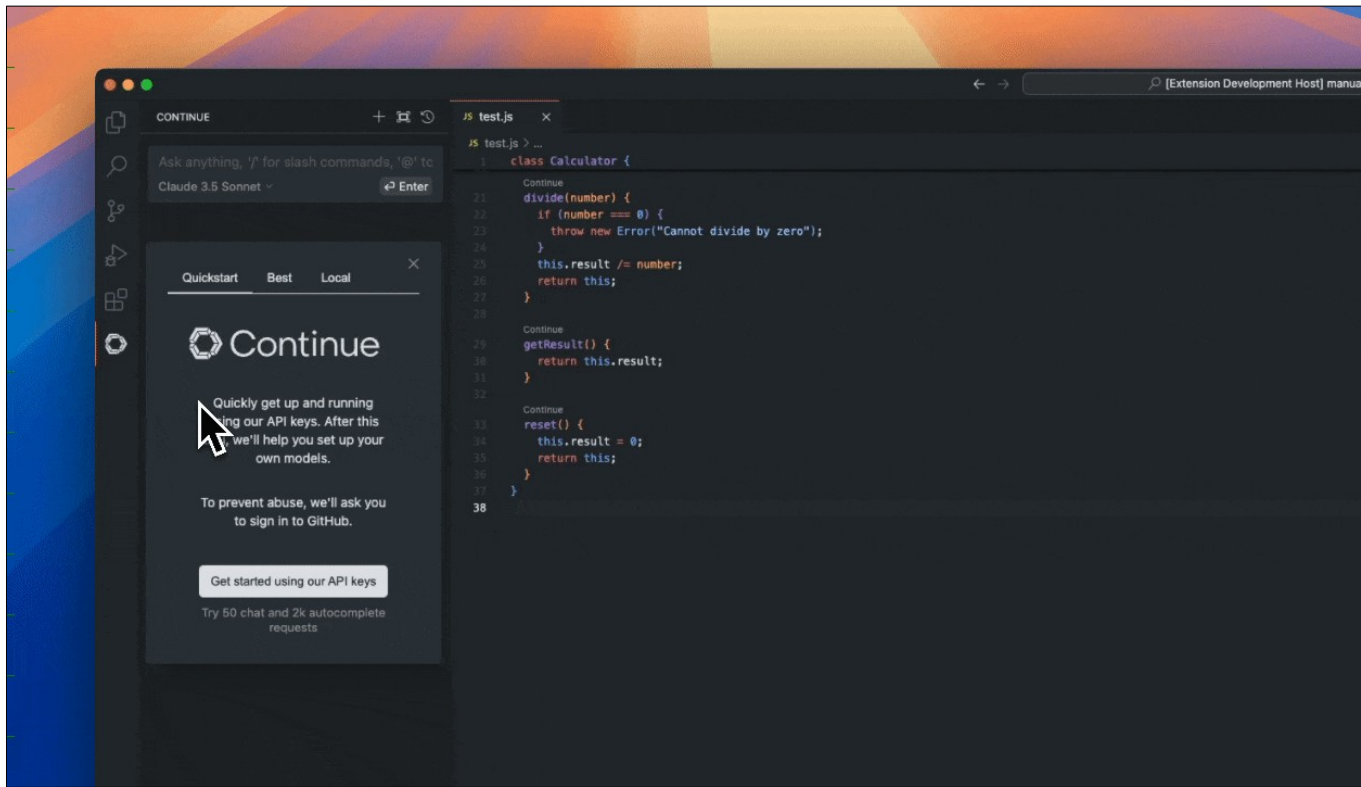
Getting Started

- To get started with vibe coding, you need to combine a model with an ***interface***
- Interfaces come in many shapes and forms, and the choice ultimately comes down to personal preference
- Common umbrella terms for these interfaces include:
 - AI coding agents or assistants
 - Agentic coding tools
 - Agent harness
- This section covers a variety of options

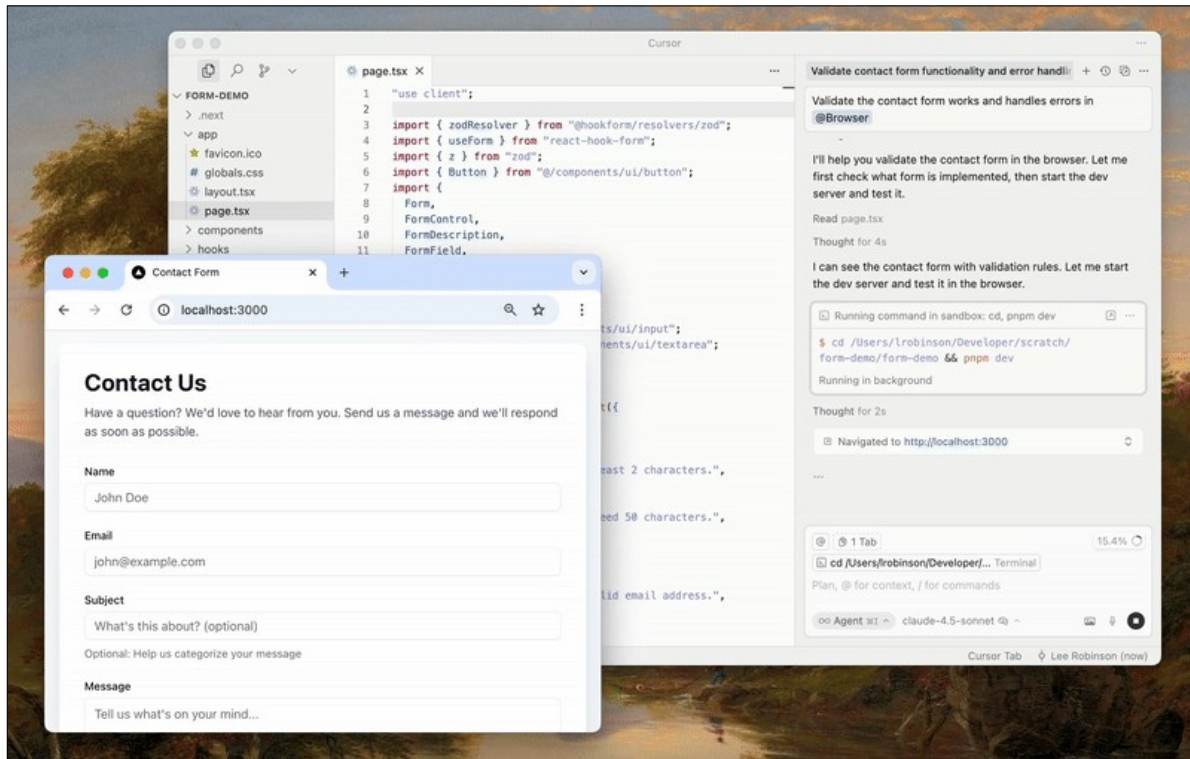
Command Line Interface (CLI) Tools



Visual Studio Code (VSCode) Plugins



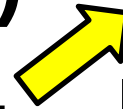
Integrated Development Environment (IDE)



Popular Open Interfaces (2026)

★ Starred 237k ▼

- [Hermes Agent](#) – “The agent that grows with you”
- [OpenCode](#) – “The open source AI coding agent”
- [Cline](#) – “an AI assistant that can use your CLI and Editor”
- [Kilo Code](#) – “The open source coding agent for building with AI in VS Code, JetBrains, or the CLI.” (Acquired by Anaconda)
- ~~[Roo-Code](#)~~ — “Your AI-Powered Dev Team, Right in Your Editor” (Archived 05/26; Pivot to Roomote)
- ~~[Continue](#)~~ — “The future of coding isn’t writing more code. It’s delegating the boring parts, so you can build the interesting stuff” (Acquired by Cursor)



HERMES-AGENT

openCode

Cline

Kilo Code

ROO CODE

Continue

Popular Closed Interfaces (2026)

- [Codex](#) – “Your new coding partner”
- [Claude Code](#) – “Your code’s new collaborator”
- [Cursor](#) – “Built to make you extraordinarily productive, Cursor is the best way to code with AI”
- [GitHub Copilot](#) – “Your AI pair programmer”
- [Lovable](#) – “Create apps and websites by chatting with AI”



CURSOR



Lovable

Model Context Protocol (MCP)

- An open standard for interfaces to access add-ons like:
 - Tools
 - Data (including databases)
 - APIs
- Some popular MCPs for vibe coding:
 - [Context7](#) for up-to-date coding documentation
 - [Playwright MCP](#) or [Chrome DevTools](#) for browser automation
 - [Comfy MCP](#) for image/video/audio/3D generation
- See the [MCP Market Leaderboard](#) for more